

Afrinext Technical Blueprint

The architecture for a multi-country African commerce and learning platform built around a double-entry ledger — stack, schema, permission model, money design, security controls and a phased build order.

Status: for review	Scope: architecture only	Code written: none	iPay: not integrated
--------------------	--------------------------	--------------------	----------------------

CONTENTS

00	The read, and one pushback
A	Technology stack
B	System architecture
C	Database schema
D	Auth & permissions
E	Financial architecture
F	Marketplace
G	Learning & video
H	Referral network
I	Delivery
J	API architecture
K	Security
L	Project structure
M	Roadmap
R	Risk register
P	What iPay needs
S	Status register
Q	Open decisions

The read, and one pushback

Afrinext is three businesses sharing one identity system. A **marketplace** (physical goods, digital goods, services), a **learning platform** (paid video and documents with entitlement), and a **money system** (wallet, commissions, settlement, payouts). Delivery and the referral network are not separate products — they are two more sources of ledger events.

That framing decides the architecture. The ledger is the spine; everything else is a producer of financial events that the ledger records. A course purchase, a delivered parcel and an ambassador commission are the same shape of thing once they reach the money layer. If that layer is designed last, every module above it grows its own private notion of "balance" and the platform becomes unauditable.

The "one person, one account, many roles" principle is right and it is cheap to honour, provided one rule holds from the first migration: **there is no role column on the user**. Roles are rows, they are scoped to a resource, and they are additive.

The pushback

The module list in the brief is a two-to-three-year product. Built as specified — all modules in parallel — every part arrives shallow, and the hardest, least reversible part (money) gets exercised last, by which point the schema is load-bearing for a dozen features.

RECOMMENDATION

Launch one vertical: digital products and courses, one country, one payment rail.

WHY	It exercises identity, roles, stores, catalog, checkout, payments, ledger, commissions, wallet, withdrawals, entitlement and protected video — every hard part except logistics — with no inventory, no couriers, no cash handling and near-instant fulfilment. It is the shortest path to a real settled transaction, which is the only thing that proves the platform works.
INSTEAD OF	Launching the full marketplace, delivery network and creator marketplace together.
TRADE-OFF	Physical goods and delivery slip to a second launch, roughly five weeks of work that starts later rather than never. Ambassadors have a thinner catalog to promote at day one.
REVERSIBLE?	Yes. Nothing in this blueprint's schema or module boundaries assumes the narrow scope — the tables for physical goods and delivery are designed here, just not built first.

This is a recommendation, not an assumption I have baked in. If you want physical goods at launch, say so and I will reorder the roadmap in section M — the architecture does not change, only the build order.

What is in the repository today

The current `Afrinext` repository holds a mobile-first PWA directory prototype: Next.js 16, Tailwind v4, an in-memory listing store, seed data, and a submission form. It is honest about what it is — there is no database, no authentication, no payments, and no money. In this plan:

ASSET	DISPOSITION
Design tokens, PWA shell, mobile navigation, card and form components	CARRIES FORWARD Becomes the seed of <code>packages/ui</code> .
Next.js 16 + TypeScript + Tailwind v4 setup, lint and build config	CARRIES FORWARD Becomes <code>apps/web</code> inside the monorepo.
<code>src/lib/store.ts</code> , <code>src/lib/seed.ts</code> , the in-memory model	DISCARD Replaced by Postgres and Drizzle.
<code>src/lib/search.ts</code> , <code>validate.ts</code> , the URL-driven filter pattern	REWRITE, KEEP THE SHAPE The pattern is right; the implementation moves to the database and to shared Zod schemas.

Technology stack

Every choice below is stated with the alternative I rejected, because several of them are genuine trade-offs rather than obvious calls.

LAYER	CHOICE	WHY THIS, OVER THE ALTERNATIVE
Language	TypeScript 5, strict	One language across web, worker and shared domain code. <code>strict</code> , <code>noUncheckedIndexedAccess</code> and <code>exactOptionalPropertyTypes</code> on from day one – retrofitting them into a 40-table codebase is miserable.
Web	Next.js 16, App Router	A marketplace lives or dies on organic search, so product, store and course pages need server rendering and stable URLs. React Server Components also ship less JavaScript to the low-end Android devices that dominate the target market. Rejected: a SPA + separate API, which doubles the deployment surface and loses SEO.
Domain logic	packages/core	Framework-free TypeScript. Next.js is transport, not the application. This is the single most important structural decision in the document – see section L.
Database	PostgreSQL 16+	Not negotiable. Real transactions, <code>SERIALIZABLE</code> isolation, CHECK and EXCLUSION constraints, deferred constraint triggers, partial indexes, <code>NUMERIC</code> , row-level locking, full-text search. The financial invariants belong in the database, not only in application code.
ORM	Drizzle ORM	See the decision block below – this one is contested.
Background jobs	pg-boss (Postgres-backed)	Lets a job be enqueued <i>inside the same transaction</i> as the ledger rows that job refers to. With Redis you get a two-phase failure: the database commits, the enqueue fails, and settlement silently never runs. Rejected: BullMQ + Redis, which is faster but buys throughput we do not have yet with a correctness hole we cannot afford. Revisit above roughly 100 jobs/second.
Auth	Better Auth + DB sessions	Phone-first OTP is a first-class requirement in this market, not an add-on. Database-backed sessions rather than stateless JWTs, because a session that controls a wallet must be revocable instantly. Rejected: Clerk and Auth0 (per-MAU USD pricing against XOF revenue), Lucia (retired upstream).
Validation	Zod	One schema per boundary – HTTP body, server action input, job payload, webhook payload – with types inferred rather than restated.
i18n	next-intl	French as the default locale, English second, locale in the URL path (<code>/fr/...</code>) so both index separately. Message catalogs live in <code>packages/i18n</code> so the worker can localise SMS and email too.
Styling	Tailwind v4 + tokens	Already in the repository, already token-driven, already dark-mode aware. No reason to change it.
Object storage	Cloudflare R2	Zero egress fees. For a platform whose core product is video in a market where bandwidth is the dominant cost, this is the difference between viable and not. S3-compatible, so the provider interface stays portable.
Video	Cloudflare Stream	Signed playback tokens, HLS adaptive bitrate, per-video access control. We store a <code>playback_id</code> , never a file URL. Alternatives worth pricing: Bunny Stream (cheaper), Mux (best tooling, most expensive). Rejected: MP4 files in a bucket – no adaptive bitrate on a 3G connection, and no way to expire access.
SMS / email	provider interface	SMS matters more than email here (OTP, order status, payout confirmations). Concrete provider chosen per country at deploy time; the interface is ours.
Observability	OpenTelemetry + Sentry	Structured logs carrying request id, actor id and ledger transaction id. A money platform without traceable financial events is not operable.

LAYER	CHOICE	WHY THIS, OVER THE ALTERNATIVE
Testing	Vitest + Playwright	Property-based unit tests on the ledger and commission engine; Playwright end-to-end on the purchase-to-payout path specifically.
Hosting	containers, Postgres in eu-west-3	Paris, not Cape Town: round-trip latency from Niamey, Bamako, Abidjan and Dakar to eu-west-3 is materially better than to af-south-1. Cloudflare in front for edge caching. Vercel works for phases 0–2 but the worker is a long-running process that does not fit its model, so plan for containers.

CONTESTED DECISION

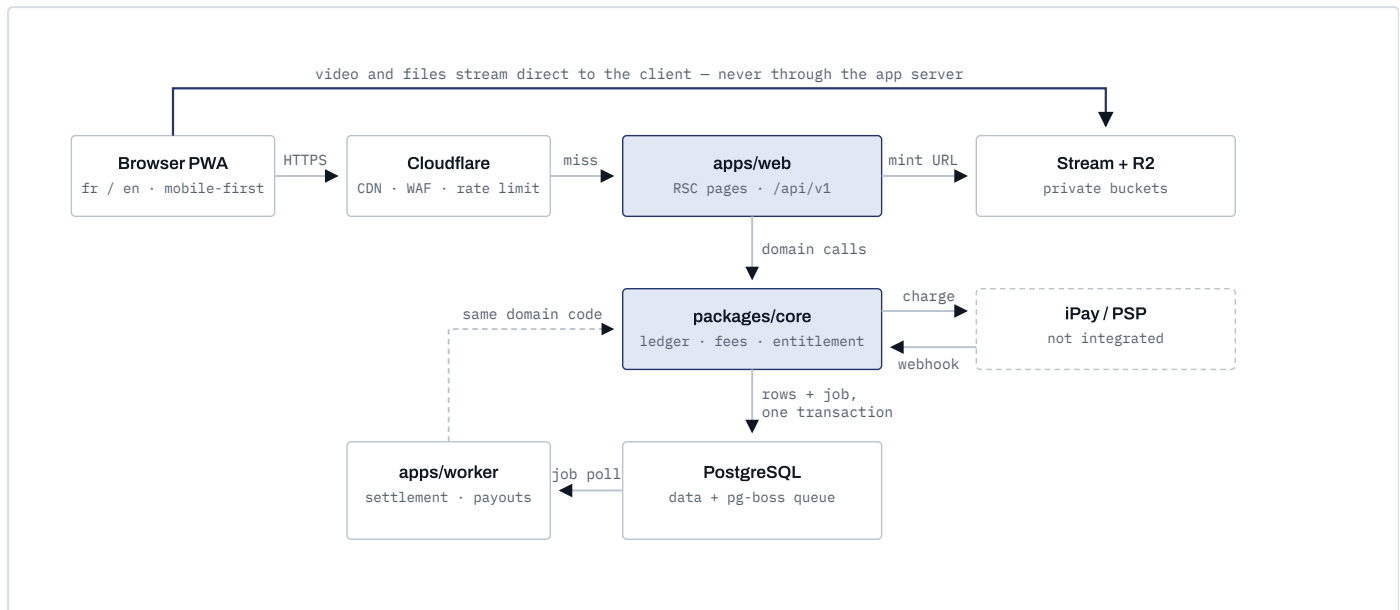
Drizzle ORM rather than Prisma

WHY	Migrations are plain SQL that a human reviews before it touches money. CHECK constraints, deferred constraint triggers and SELECT ... FOR UPDATE are expressible directly in the schema rather than through escape hatches. No query-engine binary, which matters for cold starts and for self-hosting cheaply.
INSTEAD OF	Prisma, which has better developer experience, better relational query ergonomics and far more mature migration tooling for a 40-table schema.
TRADE-OFF	More SQL written by hand, a smaller ecosystem, and a steeper ramp for developers who have only used Prisma.
SWITCH IF	The team is junior-weighted or Prisma-experienced. In that case take Prisma and write the ledger's critical paths as raw SQL — the schema in section C is unchanged either way. Tell me the team composition and I will pick.

System architecture

Four rules define the shape of the system. Everything else follows from them.

1. **Only `packages/core` touches the database.** Route handlers, server actions and job consumers all call the same domain services. There is no second path to the ledger.
2. **Jobs are enqueued in the transaction that creates their subject.** A settlement job and the ledger rows it will settle commit together or not at all.
3. **Media never passes through the application server.** The server mints a short-lived signed URL; the client fetches from storage directly.
4. **Webhooks are never processed inline.** Verify the signature, persist the raw event, return 200, and let the worker do the work. Payment providers retry aggressively and time out fast.



The database is the queue. Because pg-boss stores jobs in Postgres, a settlement job is inserted in the same transaction as the ledger entries it will later act on — the failure mode where money is recorded but its follow-up work is lost cannot occur. The dashed line from the worker means it imports `packages/core` rather than calling the web app.

Environments

Three: local (Docker Compose — Postgres, MinIO standing in for R2, a mock PSP), staging (real PSP sandbox, real Stream, anonymised data), production. Migrations run forward-only in CI with a tested rollback for each. Staging

must be able to run a full purchase-to-payout cycle before any production release touches the ledger.

Conventions that apply everywhere: UUIDv7 primary keys (time-sortable, so they index well and leak no sequence count); `timestampz` in UTC only; status enums rather than soft deletes except where retention law requires otherwise; and **money is never a bare number** — `amount_minor bigint` always travels with a `currency char(3)`.

AFRICA-SPECIFIC CORRECTNESS TRAP

XOF and XAF have **zero** decimal places. NGN (kobo) and GHS (pesewa) have two. Any code that assumes "minor units means cents, divide by 100" will be wrong by a factor of 100 across the entire UEMOA zone. The exponent comes from the `currencies` table, never from a constant.

The financial core

This is the part I would write first and change least. The rest of the schema can evolve; this cannot.

```

-- Money is always a pair, and the exponent is data, not a constant.
CREATE TABLE currencies (
  code      char(3) PRIMARY KEY,          -- XOF, XAF, NGN, GHS
  minor_unit smallint NOT NULL CHECK (minor_unit BETWEEN 0 AND 4),
  is_active  boolean NOT NULL DEFAULT true
);

CREATE TYPE ledger_account_kind AS ENUM (
  'user_available', -- withdrawable
  'user_pending',   -- earned, not yet releasable
  'platform_escrow', -- captured, owed to somebody, not yet split
  'platform_revenue',
  'psp_clearing',   -- money at the payment provider
  'payout_payable'  -- approved withdrawal, not yet paid out
);

CREATE TABLE ledger_accounts (
  id          uuid PRIMARY KEY,
  kind        ledger_account_kind NOT NULL,
  owner_type  text,                -- 'user' | 'store' | NULL for platform
  owner_id    uuid,
  currency    char(3) NOT NULL REFERENCES currencies(code),
  UNIQUE (kind, owner_type, owner_id, currency)
);

CREATE TABLE ledger_transactions (
  id          uuid PRIMARY KEY,          -- uuidv7
  kind        text NOT NULL,            -- capture | settlement | refund | payout ...
  occurred_at timestampz NOT NULL DEFAULT now(),
  idempotency_key text UNIQUE,
  reverses_id uuid REFERENCES ledger_transactions(id),
  metadata    jsonb NOT NULL DEFAULT '{}' -- order_id, payment_id, actor...
);

CREATE TABLE ledger_entries (
  id          uuid PRIMARY KEY,
  transaction_id uuid NOT NULL REFERENCES ledger_transactions(id),
  account_id  uuid NOT NULL REFERENCES ledger_accounts(id),
  direction   smallint NOT NULL CHECK (direction IN (-1, 1)),
  amount_minor bigint NOT NULL CHECK (amount_minor > 0),
  currency    char(3) NOT NULL REFERENCES currencies(code)
);

-- Append-only. The application role cannot rewrite history.
REVOKE UPDATE, DELETE ON ledger_entries, ledger_transactions FROM afrinext_app;

-- Every transaction balances, per currency. Deferred, so all entries
-- can be inserted before the invariant is checked at COMMIT.
CREATE CONSTRAINT TRIGGER ledger_must_balance
  AFTER INSERT ON ledger_entries
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION assert_transaction_balanced();

-- A cache, never the truth. Rebuilt and asserted nightly.
CREATE TABLE account_balances (
  account_id  uuid PRIMARY KEY REFERENCES ledger_accounts(id),
  balance_minor bigint NOT NULL DEFAULT 0,
  entry_count bigint NOT NULL DEFAULT 0,
  updated_at  timestampz NOT NULL DEFAULT now()
);

```

Entities by module

The brief listed candidate tables and asked me not to create them blindly. This is the filtered set, with the ones I would *not* build in phase 1 marked.

MODULE	TABLES	NOTES
Identity	users, user_identities, sessions, profiles, roles, permissions, role_permissions, role_assignments, countries, currencies, locales	<code>user_identities</code> holds one row per credential (phone, email, future OAuth) so a user can add a second login method without a schema change. <code>role_assignments</code> is scoped — see section D.
Stores	stores, store_settings, store_payout_accounts, <code>STORE_MEMBERS</code>	The <i>store</i> is the seller, not the user — orders reference <code>store_id</code> . <code>store_members</code> is deferred until multi-person stores are actually requested.
Catalog	categories, products, product_media, digital_assets, <code>PRODUCT_VARIANTS</code> , <code>INVENTORY_ITEMS</code> , <code>SERVICE_OFFERINGS</code>	One <code>products</code> table with a <code>kind</code> discriminator plus kind-specific side tables. Variants and inventory arrive with physical goods; service offerings with the services module.
Learn	courses, course_modules, course_lessons, lesson_assets, course_enrollments, lesson_progress, <code>COURSE_CERTIFICATES</code>	Certificates are a future feature in the brief and stay that way — but <code>enrollments</code> carries the completion data a certificate would need.
Commerce	carts, cart_items, checkout_groups, orders, order_items, order_fee_lines, order_events	<code>order_fee_lines</code> is the frozen commission snapshot — the reason changing a commission rate never rewrites history.
Payments	payment_intents, payment_attempts, payment_events, refunds	<code>payment_events</code> stores the raw provider payload with a <code>UNIQUE (provider, provider_event_id)</code> — this single constraint is what makes webhook replay harmless.
Ledger	ledger_accounts, ledger_transactions, ledger_entries, account_balances, settlement_holds, commission_rules, idempotency_keys	Above.
Payouts	withdrawal_requests, payout_batches, kyc_records	KYC tiers gate withdrawal ceilings. See section K.
Delivery	delivery_partners, delivery_orders, delivery_events, delivery_proofs, <code>CASH_COLLECTIONS</code>	Deferred to the physical-goods phase. <code>cash_collections</code> models the rider's cash float — see section I.
Referral	referral_programs, referral_codes, referral_attributions, referral_commissions	No <code>referral_relationships</code> tree — single level only, see section H.
Trust & ops	reviews, notifications, audit_logs, platform_settings, feature_flags, <code>DISPUTES</code> , <code>DISPUTE_MESSAGES</code>	Disputes arrive with physical goods, where they actually occur. Digital refunds in phase 1 are an admin action against an audited reversal.

Tables from the brief I would **not** create: `referral_relationships` (implied by single-level attribution), `user_roles` (superseded by scoped

`role_assignments`), and a separate `services` table (services are `products` with `kind = 'service'`).

Authentication

- **Phone + OTP is the primary method**, email + password secondary. Phone penetration far exceeds email use in the target countries, and phone is already the identifier for mobile money.
- **Database sessions, not JWTs.** An opaque token in an `httpOnly; Secure; SameSite=Lax` cookie, resolved against `sessions` on each request. A JWT cannot be revoked before it expires; a session that can move money must be killable in one query.
- Session rotation on privilege change, sign-in and password change. Shorter idle timeout for admin sessions than for shoppers.
- **Step-up re-verification** for money-sensitive actions: requesting a withdrawal, adding or changing a payout account, changing the phone number.

Authorization

Roles are scoped, additive rows. A single user simultaneously being a buyer, an instructor, a rider in Niamey and staff on someone else's store is the normal case, not an edge case.

```
CREATE TABLE role_assignments (  
  id          uuid PRIMARY KEY,  
  user_id     uuid NOT NULL REFERENCES users(id),  
  role_id     uuid NOT NULL REFERENCES roles(id),  
  scope_type  text NOT NULL,    -- 'global' | 'store' | 'course' | 'zone'  
  scope_id    uuid,             -- NULL when scope_type = 'global'  
  granted_by  uuid REFERENCES users(id),  
  granted_at  timestampz NOT NULL DEFAULT now(),  
  UNIQUE (user_id, role_id, scope_type, scope_id)  
);
```

Permissions are strings of the form `resource.action` — `product.publish`, `order.refund`, `withdrawal.approve`, `lesson.view`. Roles hold sets of them. Every check goes through one function:

```
// packages/core/authz — the only permission gate in the system.  
await authorize(actor, 'product.publish', { type: 'store', id: storeId });
```

RULE

IDOR is prevented by **scoping the query**, not by fetching a row and then checking it. Repository methods take the actor and constrain in SQL: a seller's order list is `WHERE store_id = ANY(actor.storeIds)` , never `findById()` followed by an ownership test. The second pattern works until someone forgets it once.

ADMINISTRATIVE ROLES ARE SPLIT BY LEAST PRIVILEGE

ROLE	CAN	CANNOT
support	Read users, orders, enrollments; resend receipts; open disputes	Touch the ledger, approve payouts, change commission rules
ops	Moderate catalog, verify stores, manage categories and delivery zones	Anything financial
finance	Approve withdrawals, issue refunds, post adjustments, view the ledger	Edit catalog, change permissions, alter user identities
superadmin	Role and settings administration	Approve their own withdrawal — a hard, code-level self-approval block

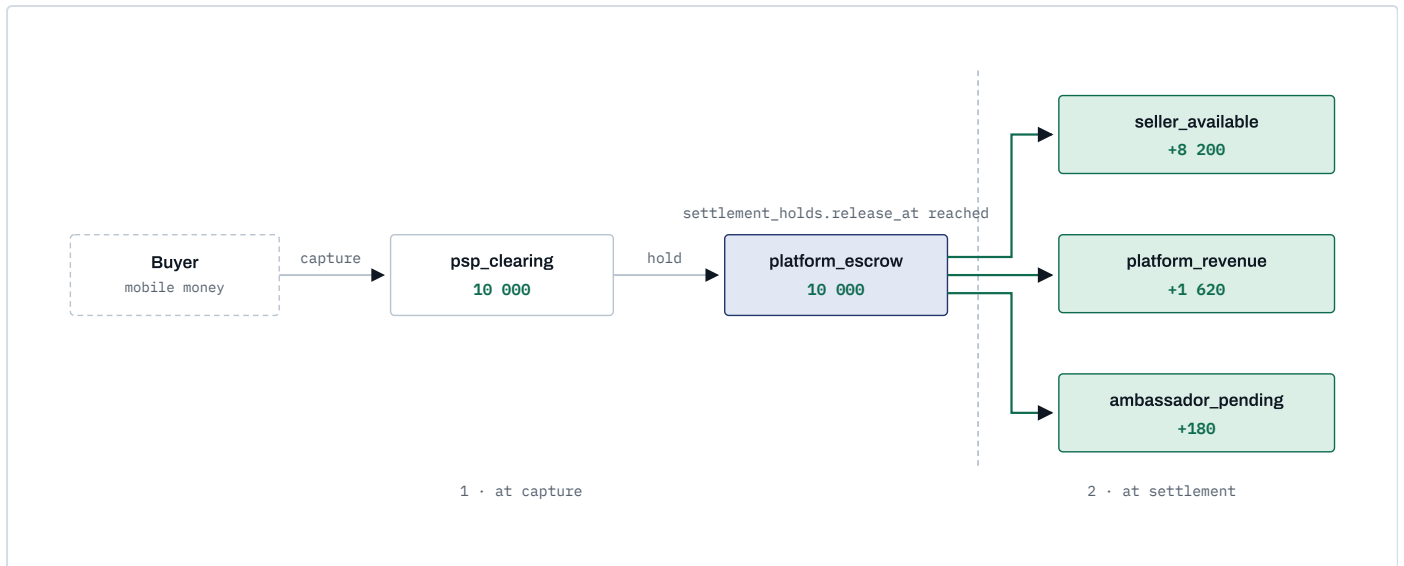
The wallet is not a balance field with rules around it. It is a **double-entry ledger**, and the balance is a derived quantity that we cache for speed and verify continuously.

The four invariants

1. **Entries are immutable.** No UPDATE, no DELETE — enforced by revoked grants, not by convention. A mistake is corrected by posting a compensating transaction linked through `reverses_id`.
2. **Every transaction balances.** Per transaction, per currency, the signed sum of entries is zero. A deferred constraint trigger rejects the COMMIT otherwise. Money cannot be created or destroyed by application code.
3. **Balances are cached, not authoritative.** `account_balances` is updated inside the same transaction under a row lock; a nightly job recomputes `SUM(direction × amount_minor)` per account and alarms on any drift.
4. **No floating point, anywhere.** `bigint` minor units and a currency code, from the database through the API to the rendered price.

Pending versus available

These are not statuses on a row. They are **two different accounts**, and "funds becoming withdrawable" is a transfer between them. That means the question "why is my money still pending?" is answerable by pointing at a specific ledger transaction and its `settlement_holds` row, rather than by reading application logic.



One 10 000 XOF course sale, at an 18% platform commission with a 10% referral program. Every arrow is one balanced ledger transaction. Note where the referral is drawn from: 180 is 10% of the *platform's* 1 800, not of the gross — so referral cost can never exceed what the platform earned on that sale. The ambassador's share lands in `pending` and is released separately once the buyer's refund window closes. For physical goods the seller's share also lands in `seller_pending` and moves to `available` on a second transaction when the delivery hold expires.

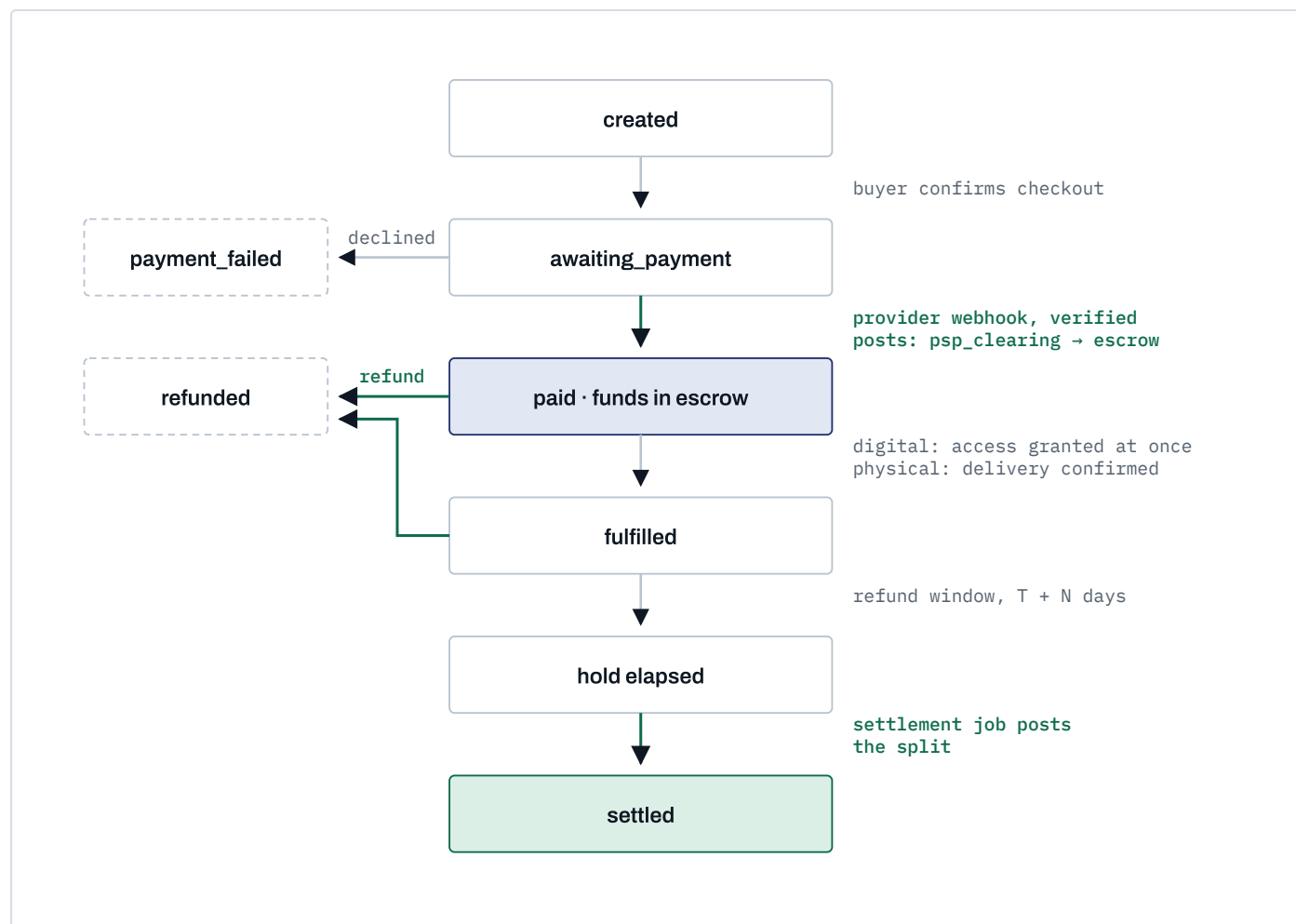
Rounding: the residual rule

Commission percentages rarely divide cleanly into whole francs. The rule is fixed and applied everywhere:

1. Compute every non-seller line (platform, referral, delivery) in minor units, rounding **half up**.
2. The seller's line is `gross - Σ(all other lines)`, never computed independently.

The split therefore re-sums to the gross by construction. No minor unit is ever created or lost to rounding, and the residual consistently favours the seller — a defensible position if it is ever questioned.

When money moves



Green transitions post ledger entries; grey ones only change order state.

Money enters escrow when the provider webhook is verified — never when the buyer returns from a redirect. A dispute does not get its own state: it *extends* the hold by pushing `settlement_holds.release_at` forward, so disputed money simply never reaches `available` while the dispute is open.

The commission engine

Rules are data. Resolution happens once, at order creation, and the result is frozen.

```
CREATE TABLE commission_rules (
  id          uuid PRIMARY KEY,
  transaction_type text NOT NULL, -- digital | course | physical | service | delivery | referral
  country_code char(2),           -- NULL = any
  category_id  uuid,              -- NULL = any
  store_id     uuid,              -- NULL = any; a negotiated rate for one seller
  rate_bps     integer,           -- basis points; 1800 = 18.00%
  fixed_minor  bigint,
  currency     char(3) REFERENCES currencies(code),
  priority     integer NOT NULL,
  effective_from timestampz NOT NULL,
  effective_to  timestampz
);
```

Resolution is most-specific-wins, broken by `priority` and then by `effective_from` — deterministic, and unit-testable in isolation. The resolved outcome is written to `order_fee_lines` with the `rule_id`, the basis and the computed amount.

WHY THE SNAPSHOT MATTERS

Raise the platform commission from 18% to 20% next quarter and **no historical order changes**. Every past settlement can still be re-derived from its own fee lines, and any seller asking "why did I receive this amount?" gets an answer that references the rule that was live at the moment they sold. A system that recomputes fees from current rules cannot answer that question truthfully.

Withdrawals

1. Request → validated against KYC tier ceiling, minimum amount, available balance, payout-account cooling period, and absence of open disputes.
2. Approved (automatically under a configured threshold, by `finance` above it) → ledger posts `user_available` → `payout_payable`. The money leaves the withdrawable balance at approval, so it cannot be double-spent while the payout is in flight.
3. Payout executed through the provider → on confirmation, `payout_payable` → `psp_clearing`.
4. On failure, a compensating transaction returns the amount to `user_available` and the request is marked failed with the provider's reason.

One products table, three kinds

`products.kind ∈ {physical, digital, service}`, with kind-specific side tables (`product_variants` + `inventory_items`, `digital_assets`, `service_offerings`). Not three separate tables — because carts, order items, reviews, search, favourites and fee resolution all need a single polymorphic target, and a three-table design forces a nullable triple foreign key into `order_items` that every query then has to unpack.

What differs between kinds is **fulfilment and hold policy**, and that lives in one place:

KIND	FULFILLED WHEN	DEFAULT HOLD
digital	Payment verified	24 h
course	Enrollment created	72 h
service	Buyer confirms, or auto after N days	7 d
physical	Delivery confirmed with proof	7 d

These are defaults in `platform_settings`, overridable per country and per category from the admin console — not constants in code.

Stores are the seller

Orders reference `store_id`, never `user_id`. This costs nothing now and buys three things later: a user can run more than one store; a store can gain staff without reshaping the order history; and a store can be transferred or suspended without touching the person's account, wallet or other roles.

Multi-store carts

A buyer will put a course from Dakar and a bag of shea butter from Bamako in one cart. At checkout the cart splits into **one order per store**, grouped under a `checkout_group_id`, paid by a single payment intent. Each order then settles independently on its own timeline — the course in 72 hours, the shea butter when it is delivered. Attempting to keep them as one order forces every fulfilment and refund decision to be all-or-nothing.

Search

Postgres full-text search with a generated `tsvector` column, plus `pg_trgm` for typo tolerance, and a French text-search configuration alongside English. Facets come from ordinary indexed columns. Move to Meilisearch or Typesense when the catalog passes roughly 100 000 rows or when faceted search gets heavy — not before, and not to Elasticsearch, which is an operational burden this team should not carry at launch.

Reviews

Only from a verified purchase: `reviews.order_item_id` is a required foreign key, unique per buyer per item. This kills fake-review farming structurally rather than through moderation, which matters when reviews feed seller ranking and seller ranking feeds income.

Structure is conventional — `courses` → `course_modules` → `course_lessons` → `lesson_assets`, with lesson kinds `video` | `pdf` | `text`. The interesting parts are entitlement and bandwidth.

Entitlement is checked at token-mint time, on every request

1. The player asks `GET /api/v1/learn/lessons/:id/playback`.
2. The server resolves the actor, checks `course_enrollments` for a live entitlement (and `access_expires_at` if the course is time-limited), and calls `authorize(actor, 'lesson.view', {course})`.
3. Only then does it mint a **signed Stream token, scoped to that user, valid 2–5 minutes**, and return an HLS manifest URL.
4. The player refreshes the token mid-playback as it nears expiry.
5. Every mint is written to `audit_logs` — which also gives instructors real "who watched what" analytics for free.

NEVER

Paid course content never has a permanent public URL. Not in a public bucket, not behind an unguessable path, not with a long-lived signed URL. An unguessable URL is a password that gets forwarded on WhatsApp.

Downloads are a per-asset instructor choice

`lesson_assets.downloadable boolean`, set by the instructor per asset. When true, downloading issues a separate short-TTL signed URL, logged, and rate-limited per user. When false, no download URL is ever minted — the flag is enforced server-side at mint time, not by hiding a button.

What is realistic about protection

BE HONEST WITH INSTRUCTORS

Signed short-lived URLs plus a burned-in overlay carrying the viewer's phone number make casual re-sharing traceable and inconvenient. They do **not** prevent a determined screen recording. Genuine prevention requires Widevine/FairPlay DRM, which is expensive and out of scope here. Instructors should be told this plainly rather than sold an assurance the platform cannot honour.

Progress and resume

`lesson_progress(enrollment_id, lesson_id, position_seconds, furthest_seconds, completed_at)` , written by a heartbeat every ~15 seconds, throttled, and flushed on pause and on unload via `sendBeacon` .

`course_enrollments.last_lesson_id` gives cross-device resume: the learner picks up on their phone exactly where they stopped on a shared laptop.

Keeping `furthest_seconds` separate from `position_seconds` means scrubbing backwards does not destroy completion state.

Built for the actual network

- An encoding ladder that **starts at 240p**, not at 720p. Adaptive bitrate matters far more here than peak quality.
- An audio-only rendition for lecture-style courses, selectable by the learner to save data.
- Progress writes are a few dozen bytes and batched — a learner should never lose their place because the heartbeat could not get through.
- Where the instructor permits downloads, offline viewing is the difference between a finished course and an abandoned one.

Afrinext Network (referrals)

The brief's principle — pay for real economic activity, never for recruitment — is enforced structurally here, not by policy text.

Attribution

- An ambassador gets a code; the link is `/r/:code`, which sets a first-party cookie (90 days) and redirects.
- At signup the cookie is consumed into `referral_attributions(referrer_user_id, referred_user_id, program_id, attributed_at, expires_at)`, with `UNIQUE(referred_user_id)`.
- **First touch, immutable.** The first ambassador to introduce someone keeps the attribution. Last-touch invites a scramble to overwrite each other's referrals just before a purchase.

Qualification

A commission is created only when a **settled ledger transaction** of a configured type exists — not at signup, not on profile completion, not on "becoming active". There is no code path that pays for a registration, because commissions are computed by a job that reads settled transactions and has no other input.

CONFIGURABLE PER PROGRAM	EXAMPLE
Basis	platform_revenue
Rate	1000 bps (10%)
Duration from attribution	365 days
Per-referral lifetime cap	150 000 XOF
Eligible transaction types	course, digital, physical, service
Country scope	NE, ML, BF ...

Single level only — no downline of a downline

WHY	Multi-level structures are where recruitment-scheme regulatory attention concentrates, and they contradict the brief's own stated principle. One level keeps the model explainable to a regulator in one sentence: "we share part of our commission with whoever introduced the seller."
INSTEAD OF	Two or more tiers, which measurably increases ambassador enthusiasm and measurably increases legal exposure.
TRADE-OFF	Slower ambassador-network growth than a tiered scheme would produce.
NOTE	Because the commission basis is platform revenue, the payout is bounded by definition — the platform can never owe more in referral than it earned on the sale. That guardrail is what makes the program safe to run at all, and it disappears if the basis is ever changed to gross.

Fraud controls

- Self-referral detection across phone number, device fingerprint, payout account and payment instrument.
- Referral commissions land in `ambassador_pending` and release only after the underlying order clears its refund window — so a wash sale that is refunded never pays out.
- Velocity limits per ambassador, with automatic hold and review above configured thresholds.
- Circular attribution (A refers B refers A) rejected at write time.

Deferred to the physical-goods phase, but designed now because it constrains the order state machine.

STATE MACHINE

requested → offered → accepted → picked_up → in_transit → delivered , with failed and cancelled as terminal branches. Every transition writes a delivery_events row with actor, timestamp and GPS point — the audit trail is the record, the current status is a cached projection of it.

DISPATCH

Offers broadcast to eligible partners in the zone; first acceptance wins via a conditional update (UPDATE ... WHERE status = 'offered') so two riders can never both hold the same job. Unaccepted offers expire and re-broadcast with a widening radius.

PROOF OF DELIVERY

A short OTP shown in the buyer's app and entered by the rider, plus an optional photo and the GPS fix at completion. The OTP is what flips the order to fulfilled and starts the settlement hold — a rider cannot self-declare a delivery into existence.

RIDERS LOSE SIGNAL CONSTANTLY

The rider interface queues state transitions locally and replays them with idempotency keys when connectivity returns. A transition applied twice must be a no-op; a transition applied out of order must be rejected by the state machine, not silently accepted.

CASH ON DELIVERY IS A BIGGER PROBLEM THAN IT LOOKS

COD is the dominant payment method for physical goods across much of the region, and it inverts the money flow: the rider collects cash, so the platform is owed money by the rider rather than holding the buyer's funds. That needs a cash_collections liability account per rider, a deposit and reconciliation process, float limits, and a real operational routine — it is a project, not a checkbox. **My recommendation is to launch physical goods prepaid-only and add COD as its own phase with its own design.**

Two entry styles, one implementation. **Server Actions** for first-party form mutations (fewer round trips, progressive enhancement) and **versioned REST** at `/api/v1/*` for everything a future native app, an ambassador tool or a partner integration would consume. Both call the same `packages/core` services. Logic is never duplicated between them, and neither one is allowed a private path to the database.

DOMAIN	REPRESENTATIVE ENDPOINTS
auth	POST <code>/auth/otp/request</code> · <code>/auth/otp/verify</code> · <code>/auth/session</code> · DELETE <code>/auth/session</code>
me	GET <code>/me</code> · PATCH <code>/me</code> · GET <code>/me/roles</code> · GET <code>/me/orders</code> · GET <code>/me/enrollments</code>
stores	POST <code>/stores</code> · GET <code>/stores/:slug</code> · PATCH <code>/stores/:id</code> · GET <code>/stores/:id/analytics</code>
catalog	GET <code>/products</code> · GET <code>/products/:slug</code> · POST <code>/stores/:id/products</code> · POST <code>/products/:id/publish</code>
search	GET <code>/search?q&kind&category&country&cursor</code>
cart · checkout	GET/POST <code>/cart/items</code> · POST <code>/checkout</code> · GET <code>/checkout/:groupId</code>
orders	GET <code>/orders</code> · GET <code>/orders/:id</code> · POST <code>/orders/:id/cancel</code> · POST <code>/orders/:id/refund</code>
payments	POST <code>/payments/intents</code> · GET <code>/payments/:id</code> · POST <code>/webhooks/:provider</code>
wallet	GET <code>/wallet</code> · GET <code>/wallet/transactions</code> · POST <code>/wallet/withdrawals</code> · GET <code>/wallet/withdrawals/:id</code>
learn	GET <code>/courses/:slug</code> · GET <code>/learn/lessons/:id</code> · POST <code>/learn/lessons/:id/progress</code> · GET <code>/learn/lessons/:id/playback</code>
delivery	GET <code>/delivery/offers</code> · POST <code>/delivery/:id/accept</code> · POST <code>/delivery/:id/transition</code>
referrals	GET <code>/referrals/me</code> · GET <code>/referrals/commissions</code> · GET <code>/r/:code</code>
admin	<code>/admin/*</code> – mirrors the above under <code>admin.*</code> permissions

CONVENTIONS

- **Idempotency-Key** is required on every POST that moves money. The key, the request hash and the resulting transaction id are stored; a replay returns the original response rather than acting twice.
- Cursor pagination everywhere (`?cursor=&limit=`). Offset pagination on a growing catalog silently skips and duplicates rows.
- Responses are `{ data, meta }`; errors are `{ error: { code, message, fields } }` with **stable machine-readable codes** — clients must never branch on message text, which is localised.
- Money crosses the wire as `{ amountMinor: 8200, currency: "XOF" }`. Never a formatted string, never a float.
- Rate limits by route class: strictest on OTP send and verify, then auth, then write, then read.
- Webhooks return `200` as fast as possible after signature verification and persistence. All work happens in the worker.

CONTROL	WHERE IT IS ENFORCED
Authentication	Opaque session tokens in <code>httpOnly; Secure; SameSite=Lax</code> cookies, resolved server-side. Rotated on sign-in and privilege change. Argon2id for the password path.
Step-up verification	Fresh OTP required for withdrawal requests, payout-account changes and phone changes — regardless of session age.
Authorization	One <code>authorize()</code> choke point in <code>packages/core/authz</code> . No route handler makes its own decision.
IDOR	Prevented by scoping queries to the actor in SQL, not by post-fetch ownership checks.
Input validation	Zod at every boundary; database CHECK and FK constraints as the last line. Frontend validation is a convenience and is never trusted.
Rate limiting	Per IP and per user, at the edge and in the app. Hardest limits on OTP send/verify, login, withdrawal, and playback-token minting.
Webhook integrity	Signature verified against the raw body before parsing; timestamp freshness window; <code>UNIQUE (provider, provider_event_id)</code> makes replay a no-op.
File access	Private buckets only. Short-TTL signed URLs minted after an entitlement check. Every mint audited.
Payment card data	Never touched. Provider-hosted collection only, which keeps PCI scope at SAQ-A. This is a deliberate architectural constraint, not an oversight.
Audit	Append-only <code>audit_logs</code> for every admin action and every financial state change: actor, action, target, before/after, IP, user agent, request id.
Fraud	Velocity checks, device fingerprinting, a cooling period before first withdrawal to a newly added payout account, and automatic hold-and-review thresholds.
Separation of duty	No user can approve a withdrawal to an account they control. Enforced in code, not in policy.
Secrets	Environment only, distinct per environment, rotated. Sandbox and production provider credentials never coexist in one config.
Data protection	PII minimised and encrypted at rest; retention policy per data class; export and deletion paths built for Nigeria's NDPR, Ghana's Data Protection Act and UEMOA national regimes.

EXPLICITLY OUT OF SCOPE, AND NAMED AS SUCH

DRM for video (Widevine/FairPlay). A third-party penetration test — which should be scheduled and budgeted *before* public launch, not after. Formal PCI-DSS certification beyond SAQ-A. Automated AML transaction-monitoring beyond threshold rules.

pnpm workspaces plus Turborepo. The directory layout is not cosmetic — it is what physically prevents domain logic from leaking into the framework.

```

afrinext/
├─ apps/
│  ├─ web/           Next.js 16 – pages, route handlers, server actions
│  └─ worker/        long-running pg-boss consumers
├─ packages/
│  ├─ core/          framework-free domain logic – the application
│  │  ├─ money/       Money type, minor units, rounding, formatting
│  │  ├─ ledger/      posting, balances, reconciliation
│  │  ├─ commissions/ rule resolution, fee snapshots
│  │  ├─ orders/      order + settlement state machines
│  │  ├─ payments/    PaymentProvider interface + registry
│  │  ├─ learn/       entitlement, playback tokens, progress
│  │  ├─ delivery/
│  │  └─ referrals/
│  │     └─ authz/    permissions, authorize()
│  ├─ db/            Drizzle schema, SQL migrations, seeds
│  ├─ providers/     psp-ipay, psp-manual, storage-r2, video-stream, sms, email
│  ├─ ui/            design system (seeded from today's prototype)
│  ├─ i18n/          fr / en message catalogs, shared with the worker
│  └─ config/        tsconfig, eslint, tailwind preset
├─ docs/
│  └─ architecture/  this document
│  └─ adr/           architecture decision records, numbered
│  └─ runbooks/      reconciliation drift, stuck payout, failed webhook
└─ infra/           docker, compose, CI, deploy

```

THE RULE THAT MAKES THIS WORTH DOING

`packages/core` may import `packages/db`. It may **not** import anything from `next`, and this is enforced by an ESLint boundary rule in CI, not by discipline. The consequence: the ledger and commission engine are testable in milliseconds without a server, the worker runs identical business logic to the web app, and extracting a standalone API service later is a packaging change rather than a rewrite.

Development roadmap

Phases are ordered by *irreversibility*, not by visibility. The money spine comes before any product feature, because it is the thing that cannot be retrofitted. Durations assume two to three engineers and are estimates, not commitments.

PHASE	SCOPE	EXIT CRITERION – THE THING THAT MUST DEMONSTRABLY WORK	EST.
P0 Foundations	Monorepo, Postgres, Drizzle schema v1, auth (phone OTP + email), scoped RBAC, i18n, design system extraction, CI, observability skeleton	A user registers by phone, signs in, switches language. CI runs typecheck, lint, tests and migrations forward and back on every commit.	2 wk
P1 Money spine	money , ledger , commission engine, idempotency, settlement holds, reconciliation job, admin ledger viewer	Property-based tests over 10 000 randomised operations prove every transaction balances and the cached balance equals the sum of entries. An injected drift is caught by the reconciliation job. No product feature is built before this passes.	3 wk
P2 Stores & catalog	Store creation, digital products and courses as products, categories, media upload to R2, public storefront and product pages, search	A seller publishes a paid digital product that renders server-side at a public URL with correct metadata and appears in search.	3 wk
P3 Cycle closed, no PSP	Cart, checkout, orders, fee snapshots, and a ManualTransferProvider where an admin confirms receipt	The complete economic cycle runs end to end without any payment integration: buyer orders → admin confirms → ledger settles → seller sees earnings → requests withdrawal → finance marks it paid. This de-risks everything that depends on iPay.	3 wk
P4 iPay	Real provider implementation behind the existing interface	In sandbox: charge created, webhook signature verified, status query reconciled, refund executed, replay proven harmless. GATED Cannot start until the documentation and sandbox credentials in section P are in hand.	unknown
P5 Afrinext Learn	Modules, lessons, Stream upload, entitlement, signed playback, progress and resume, download flag, learner interface	An automated test proves a paid lesson is unplayable without entitlement, and that resume works across two devices for one learner.	4 wk
P6 Wallet & payouts	Wallet interface, KYC tiers, withdrawal flow with step-up auth, payout batches, finance console	Full withdrawal lifecycle including a provider failure that correctly reverses back to available balance.	3 wk
P7 Network	Referral programs, attribution, qualification, commissions, ambassador dashboard, fraud checks	Tests prove a commission can only originate from a settled transaction, that self-referral is blocked, and that a refunded order pays nothing.	3 wk
P8 Physical & delivery	Variants, inventory, shipping, delivery partner interface, state machine, proof of delivery, delivery earnings	A physical order settles only on OTP-confirmed delivery, and a rider replaying a queued transition twice changes nothing.	5 wk
P9 Services & creators	Service offerings, booking and request flow, creator categories and portfolios	A service settles on buyer confirmation, and on auto-confirmation after the configured window.	3 wk
P10 Hardening	Load test, penetration test, runbooks, restore drill, compliance review, admin settings completeness	A database restore is performed from backup against a stopwatch, and the recovery time is written down.	3 wk

The admin console is **not** a phase. Each phase ships the admin surface for what it built; an admin console deferred to the end is an admin console that never gets finished.

To the recommended launch vertical (P0–P3, P5, P6, plus P4 whenever iPay unblocks): roughly 18 weeks. Full scope as briefed: roughly 32 weeks. If iPay stalls, P3's manual rail means you can onboard real sellers and take real money by bank transfer while it is resolved.

RISK 1 – THE ONE THAT CAN STOP THE PROJECT

Holding customer funds may be a regulated activity. In the UEMOA zone, taking money from a buyer, holding a seller's balance and disbursing it can fall under BCEAO e-money and payment-services rules; Nigeria and Ghana have their own regimes. This is not an engineering problem and I cannot solve it in the architecture — but the architecture can be shaped to reduce exposure by keeping funds at the licensed provider and having Afrinext record *entitlement* rather than custody. **Get local counsel before P6 (payouts) ships.** Please tell me who is advising on this.

RISK	SEV	MITIGATION
Ledger correctness — a bug here is money, and it is discovered by users	HIGH	Double-entry with database-enforced balancing, immutability by revoked grants, property-based tests, nightly reconciliation with alarms, ledger built in P1 before anything depends on it
iPay is an unknown — the API may lack payouts or refunds entirely	HIGH	Provider abstraction plus the manual rail in P3, so no other phase is blocked. Section P lists exactly what is needed to remove this risk
Referral fraud and scheme perception	MED	Single level, commission drawn from platform revenue only, qualification from settled transactions only, pending-until-refund-window, self-referral detection
Scope — the brief is a 32-week product and enthusiasm compounds	HIGH	The launch vertical in section 00; phases with hard exit criteria; nothing starts until the prior phase's criterion demonstrably passes
KYC and AML on withdrawals	MED	Tiered KYC with withdrawal ceilings per tier; identity verification before the first payout; threshold-based holds. Provider choice still open
Mobile money reversal semantics differ from cards and vary by operator	MED	Conservative hold windows, configurable per country and per provider; never assume card-like chargeback behaviour
Video bandwidth cost and learner data cost	MED	R2 zero-egress, ladder starting at 240p, audio-only rendition, permitted offline download
Latency from West Africa	LOW	Postgres in <code>eu-west-3</code> , Cloudflare edge caching. Measure real round-trip from Niamey, Lagos and Accra in P0 and revisit if the numbers disagree with the assumption
Cash on delivery reconciliation	MED	Recommend launching physical goods prepaid-only; COD gets its own phase and its own liability model
Cross-jurisdiction data protection	MED	PII minimisation, encryption at rest, per-class retention, export and deletion paths built in P0 rather than bolted on
Operating a money system on-call	MED	Runbooks written in the phase that creates the failure mode; reconciliation alarms route to a human; restore drill rehearsed in P10

What the iPay integration needs

NOTHING IS INTEGRATED

No iPay endpoint, parameter, signature scheme or webhook payload appears anywhere in this document, and none will be written until the official documentation and sandbox credentials are in hand. What exists in the plan is an *interface* that iPay will be implemented behind. Any code claiming otherwise would be fiction.

The abstraction Afrinext codes against:

```
interface PaymentProvider {
  createCharge(input: ChargeInput): Promise<ChargeResult>;
  getCharge(providerRef: string): Promise<ChargeStatus>;
  verifyWebhook(rawBody: Buffer, headers: Headers): Promise<VerifiedEvent>;
  refund(providerRef: string, amount: Money): Promise<RefundResult>;
  createPayout?(input: PayoutInput): Promise<PayoutResult>; // optional – may not exist
  getPayout?(providerRef: string): Promise<PayoutStatus>;
}
```

`createPayout` is optional on purpose. If iPay has no disbursement API, payouts run as an operational batch and the platform still works — that fact should be established early, because it changes the finance team's workload substantially.

REQUIRED FROM THE OFFICIAL IPAY DOCUMENTATION, BEFORE P4 CAN BEGIN

1. Base URLs for sandbox and production.
2. Authentication scheme — API key, HMAC, OAuth client credentials — and where credentials belong in a request.
3. Charge initiation: request and response schema, and the supported channels per country (Orange Money, MTN MoMo, Moov, Wave, Airtel, card, bank).
4. Flow type: hosted redirect, inline widget, or USSD push. This changes the checkout UI, not just the backend.
5. Status query endpoint and the complete status enumeration, including every terminal and non-terminal value.
6. Webhooks: how the URL is registered, payload schema, signature algorithm and header name, timestamp tolerance, and retry policy.
7. Currency handling and the minor-unit convention — specifically whether XOF amounts are sent as whole francs.
8. Whether a disbursement or payout API exists, and its settlement timing.
9. Refund API and its constraints — partial refunds, time limits.

10. Idempotency support on charge creation, and rate limits.
11. Settlement schedule to the merchant account (T+N), and the fee structure.
12. Sandbox credentials plus test phone numbers or accounts for each channel.

Items 6, 7 and 8 are the ones that most often invalidate an assumed design. If only a subset is available now, send items 1–6 and P4 can begin against a partial implementation.

The brief asked that anything unimplemented be marked as such. This is that list, and it is complete as of this document.

ITEM	STATUS
Everything in this blueprint	NOT IMPLEMENTED Architecture only. No code from this plan has been written.
iPay payment integration	NOT IMPLEMENTED Blocked on the documentation in section P. Not designed against, not tested, not stubbed with invented shapes.
Database, authentication, wallet, ledger, courses, delivery, referrals	NOT IMPLEMENTED None exist in the repository today.
Cloudflare Stream / R2, SMS provider, KYC provider	NOT SELECTED Recommended, not procured, no accounts created, no pricing confirmed.
Latency assumption (<code>eu-west-3</code> over <code>af-south-1</code>)	UNVERIFIED Based on general routing, not on measurement from the target cities. Verify in P0.
Effort estimates in section M	ESTIMATES Two to three engineers assumed. Not commitments.
Regulatory position on holding funds	UNRESOLVED Requires local legal counsel. Outside what I can determine.
Existing repository prototype	WORKING Builds, lints, and was verified against a running server — but it is a directory demo with an in-memory store, not a foundation for this plan beyond its UI layer.

Open decisions I need from you

Answers to 1–4 change the plan materially. The rest can be settled during P0 without blocking anything.

1. **Regulatory posture.** Who is advising on whether Afrinext may hold and disburse user funds in the launch country? This gates P6 and could reshape the wallet's legal model.
2. **Launch scope.** Do you accept the digital-and-courses-first vertical, or do physical goods and delivery ship at launch? This reorders roughly nine weeks of work.
3. **iPay status.** Is the contract signed, and can you obtain the documentation and sandbox credentials listed in section P? If not, P3's manual rail becomes the launch payment method and we should plan for that openly.
4. **Referral levels.** Confirm single-level. If you want tiers, that needs the legal review in item 1 to cover it explicitly.
5. **Launch country and currency.** Niger and XOF assumed. Confirm, or name the first two.
6. **Team composition.** How many engineers, and what is their Prisma versus SQL experience? This settles the ORM question in section A, and it is the only reason that decision is still open.
7. **Hosting preference.** Managed and simple (Vercel + Neon, higher unit cost) or containers (Hetzner or Fly, cheaper, more operations)?
8. **Native app.** Is the PWA sufficient at launch, or is a store-distributed app required? It does not change the API design, but it changes how much weight the REST layer carries in P2 and P3.
9. **Cash on delivery.** Required at physical-goods launch, or can it be its own phase?
10. **KYC.** Which identity documents are practical to verify in the launch country, and is there a provider you already work with?

On approval, P0 and P1 are what I would start on — the monorepo, the schema, and the ledger with its property tests — because they are the parts that everything else is built on top of and the parts that are most expensive to change later. Nothing above is locked; tell me where you disagree and I will revise the blueprint before any code is written.